

READ THIS FIRST

Attach your finished exercise to your Upwork proposal. A proposal that arrives without it is not read, whatever else it says. We need the repository link, with its branches and pull requests left intact, plus the two documents on the last page.

Six hours, capped. **If we hire you, those six hours are paid.**

TECHNICAL EXERCISE

Product Engineer Full Stack

A vague brief, six hours, and every pull request you open on the way there. We are looking for someone who can point AI at a product and still own the engineering.

THE BUILD**A product**

Interpret a page of meeting notes and put something coherent on its feet.

THE HISTORY**How you got there**

Every change through a pull request you read before you merged it. We read that too.

TIME CAP

6 hours of effective work.
Hard stop.

DEADLINE

One calendar week from receipt.

AI

Required, not merely permitted.

DELIVER VIA

Attached to your Upwork proposal.

BEFORE YOU START

What we are actually testing

This is not a test of how fast you write code. We assume you will use Claude Code, Cursor, Codex or whatever you already run, and that you will use it for everything. That is how we build here, and the only way this scope fits in the time.

What we want to see is whether you can turn those tools into an engineering system. Someone who reads the pull request their agent just opened and sees, in the first minute, the thing that leaks data between tenants, the thing that becomes a migration you cannot run in three months, and the thing that is not worth arguing about. Which is why we are going to read your commit history as closely as we read your code.

THE SIX HOUR RULE

Stop at six hours. Whatever exists at the six hour mark is your submission, and we will read it assuming it took exactly six hours. There is no credit for going over, and it shows: a submission that clearly took twenty hours tells us you cannot scope.

Note your real time in the README, we take your word for it. You have one calendar week from receipt and the six hours can be split across it however you like.

CONTEXT

What we do, and where it hurts

Sellervate runs customer experience for ecommerce brands. A brand hands us their support inbox and we answer it *as them*. A customer writes to what they believe is the scooter company, and one of our specialists replies in that brand's voice, following that brand's procedures, from that brand's helpdesk. The customer never learns we exist, and that is the point.

What makes it hard to manage is that one specialist covers several brands in a day, and every brand wants something different. For a technical product a good reply diagnoses the problem before it offers a refund, because half the complaints are people using the thing wrong. For a commodity supplier a good reply is fast, exact and three lines long. The same paragraph can be excellent for one brand and completely wrong for the next.

So the quality of those replies *is* the product, and it is the only part of us the client ever sees. Today we check it the way you would guess: a team lead opens the shared inbox, skims what went out yesterday, and pings someone on Slack when something looks off. That stops working around the fourth brand, and it leaves no record, so we cannot coach from it and we cannot answer a client who asks whether we are getting better.

We want an internal tool for this. There is no specification, no screen list and no use cases.

TWO DIFFERENT THINGS CALLED AI

You will use AI to build this. That is required, assumed, and the last we will say about it.

The thing you build is not an AI product. It is a tool for two people to exchange judgement about work that already happened: a team lead records what they thought of a reply, the specialist reads it back. Nothing in it needs a model, and we are not asking you to add one. Spend two of your six hours on automatic scoring and you will have built the wrong thing well.

But if you think a model belongs somewhere, say so. Catch yourself thinking "an agent should triage this queue" and write that in `DECISIONS.md` instead: where it would sit, what it would do, what would have to be true before you trusted it. A paragraph, not a feature. Where you would and would not reach for one tells us more than the wrapper would.

BEFORE YOU START DESIGNING ANYTHING

What a Monday morning looks like with this thing

We are not going to tell you what to build. But it is worth knowing what kind of object this is, so you spend your six hours on the interesting decisions rather than on guessing the genre.

Marta leads three brands. She opens the tool and sees what her specialists sent out yesterday, thirty or so replies. She reads five of them, and on each one she records how good it was and what was off about it: too slow, wrong tone for that brand, answered a different question than the one asked, technically correct but would not have stopped the customer writing in again.

Two weeks later that brand asks how things are going. She opens the brand and can answer: here is the trend, here is what we keep getting wrong, here is what we changed about it.

Meanwhile Dani, one of her specialists, logs in and sees his own scores and what Marta wrote on them. He does not see anybody else's.

WHAT IT IS NOT

Nobody talks to a customer inside this tool. It is not a helpdesk, not an inbox and not a ticketing system. The replies already went out. This is where somebody looks at them afterwards and decides whether they were any good.

AND THE REPLIES THEMSELVES ARE YOURS TO INVENT

Everything in your database is made up by you. Nobody is going to hand you a dataset, and asking for one is not a question we will answer.

Worth knowing, though: this is a product for *reading what somebody wrote*. If the replies are lorem ipsum then nobody can see it work, including you. Six plausible support replies across two brands that clearly do not sound alike, one of them obviously bad, beats two hundred rows of filler. An agent will write them in a couple of minutes.

YOU MAY FAKE THE LOGIN. YOU MAY NOT FAKE WHO CAN SEE WHAT.

Authentication can be stubbed. A user switcher in the corner is fine. Wiring up a real login is the most expensive thing in this exercise and it tells us nothing we want to know, so we would rather you spent that time on the product.

Authorisation cannot be stubbed. Whatever your app decides a specialist is allowed to see has to be enforced on the server, not in the browser. Hiding a button is not enforcing anything. If we switch to a specialist and ask the API directly for another brand's data, it should say no.

Which layer you enforce it in is your decision, and we would like to hear it. There is more than one defensible answer here, including more than one at once. Tell us in `DECISIONS.md` where you put it, why there, and what real authentication would take on top.

THE ONLY BRIEF THERE IS

What we know

Loose sentences somebody wrote down during one meeting with the team lead who would use this. There are no minutes, no follow up document and nobody available to answer questions.

- “ I can tell in three replies whether someone has actually read the brand's procedures. But I get through maybe five a day, and we send hundreds.
- “ For the scooter brand a good reply diagnoses before it offers a return. For the packaging brand it just has to be fast and exact. Same person writing both, completely different job.
- “ Getting the tone slightly wrong is annoying. Telling a customer the wrong thing about their own product is how we lose the account.
- “ One of them was closing tickets without checking the order history for about a month. We found out because the brand found out.
- “ Most specialists cover two or three brands. I cover four of the six and Nuria covers the rest, so neither of us sees the whole picture.
- “ A specialist should see their own scores and whatever I wrote on them. Not everybody else's.
- “ When somebody new joins we sit them down with the good ones and the bad ones and explain the difference. Right now that means me digging through the inbox the night before.
- “ Every quarter I tell a brand we are improving. I would rather show them the number than say the sentence.
- “ Eventually this should pull the replies out of the helpdesk on its own. Not now. Just do not make it impossible later.

Where these are ambiguous, and they are: decide, write the assumption down, defend it.

Three readings, all of which we would accept

The notes point in more than one direction and you cannot serve all of them in six hours. Here are three honest interpretations. Pick one, build that one properly, and tell us why in your decisions document.

IF YOU READ IT AS	WHAT YOU WOULD BUILD FIRST
A reviewing problem	The loop itself. Getting through five replies quickly and consistently is the whole product, and reporting is a list you can read.
A proof problem	The trend and the evidence. Scoring is a minimal form, and the work goes into what you can put in front of a client.
A coaching problem	The library. Scored replies become the good and bad examples you hand a new joiner, with the reasoning attached.

There are other defensible readings. These are examples of the size of decision we are asking you to make, not a menu.

THE ASSIGNMENT

Design and build the first version of this tool, in the open, one reviewed pull request at a time.

Part 1 • Build ~4 H 30

Interpret the notes and build a first version you would be comfortable putting in front of a team lead on Monday. Not a feature list: a defensible reading of the problem, and a coherent product built around it. Deciding what *not* to build is most of the work, and that decision is what we read first.

Go for something that stands up end to end rather than one beautiful screen. A complete product with rough edges beats a perfect fragment, and rough edges are expected in six hours. **One role's journey working properly is a legitimate choice** if you tell us it was a choice.

HOW WE WANT YOU TO WORK, AND THIS PART IS NOT OPTIONAL

Unless you have a better way of working, in which case use it and tell us why, this is the shape we expect and the shape we will be reading:

1. A branch per piece of work, small enough that the pull request is readable in five minutes.
2. Your agent writes the code and opens the pull request.
3. **You review it on the pull request, in writing, before you merge.** What is wrong, what you are letting go and why. If it is genuinely clean, say so in one line.
4. You or the agent address the comments in follow up commits on the same branch.
5. Merge.

Do not squash. Do not rebase the history into something tidy. Do not open one enormous pull request at the end. A repository whose entire history is one commit called "initial commit" tells us nothing, and we will score it as though you had sent nothing.

We are not asking for ceremony. Four or five real pull requests across six hours is plenty, and a one line review that says "fine, merging" on a small change is a legitimate review. What we want to see is whether you read what the agent hands you before you take it.

Part 2 • Decisions ~45 MIN

DECISIONS.md, two pages maximum, no filler. Short and direct beats thorough.

- **Product.** What you think the real problem was. What you built first and why. What you left out and why. Where in this product a model would earn its place, if anywhere, and what would have to be true first. What you would ask us before a V2.
- **Architecture.** Why this shape. How you modelled the data. How authorisation is enforced, and what real authentication would take. What breaks first as this grows.
- **AI.** How you actually worked. Where the agent was right, where you overrode it, and what it took to get a good result. Paste one prompt or session excerpt you are pleased with.
- **Status.** What is finished, what is half done, what you never touched, and the order in which you would pick it up. Then name the one thing in your own repository you would flag hardest if it turned up as somebody else's pull request, and say why you left it.

TECHNICAL CONSTRAINTS

Few, but fixed

We have kept these short on purpose, but they are not negotiable, because we want to see you working in the environment you would actually be joining.

- Monorepo on **GitHub**, public, with the branches and pull requests left intact. We read them. A repo we cannot open counts as nothing sent.
- **Next.js**, App Router, TypeScript.
- **Supabase** for data: Postgres. Local via `supabase start` or a free hosted project, your call. Auth is a special case, see page 3.
- **Tailwind**. daisyUI if you like it, we use it.
- A README that takes a stranger from clone to running in under ten minutes.
- **Seed data, invented by you**. We are not sending you any and there is nothing to wait for. At least two brands, three specialists and two team leads, a way for us to be each role, and enough scored rows that an average means something.
- AI use expected. If you keep session logs or prompt files, commit them. We enjoy reading them.

DESIGN

We care about this a lot

The reference is **sellervate.com**. Do not copy it. We want evidence of intent: a type scale you chose, a colour system you can explain, empty and loading and error states that were designed rather than defaulted.

This is an internal tool, which is not a licence for it to be ugly. Internal tools are where taste actually shows, because nothing is hiding behind a marketing page.

SCORING

What we evaluate, and how much it weighs

DIMENSION	WHAT WE LOOK AT	WEIGHT
How you worked	Your pull requests and what you wrote on them. Whether you read the agent's output before taking it.	24
Problem interpretation and prioritisation	Whether you understood what they need behind what they said, and can defend what you cut.	18
Architecture and code	Structure, separation of concerns, legibility. Whether we can build on top of it.	13
Data model	Whether the entities and relations survive the growth of the product.	12
Security and tenant isolation	Whether authorisation is enforced on the server, and whether brands and roles are actually isolated.	10
Interface and visual craft	Whether it is understood without a manual, and whether it looks considered.	10
Decisions document	Clarity of reasoning. It weighs as much as a feature.	8
Ground covered in six hours	How much coherent product you get standing. Where using AI well becomes visible.	5

What we do not evaluate: exhaustive test coverage, edge case polish, deployment, or a product free of rough edges. We will run it locally. Using AI is not penalised, it is assumed.

DELIVERY

Send it as part of your Upwork proposal, not afterwards. Put the repository link in the proposal text, along with the hours you actually spent. Keep the repository public: one we cannot open counts as nothing attached.

Questions about the exercise can go in a message on Upwork before you start. Questions about the product go in your decisions document, answered by you.

ONE THING WORTH SAYING PLAINLY

This is not work we are shipping. The tool described here is not on our roadmap and nothing you build will end up in our product. We are testing judgment, not collecting free features.

AFTER YOU SEND IT

What happens next

We read your pull requests and your decisions document first, and run the thing locally second. If it goes further we book **45 minutes**. No live coding. We will walk through your decisions, two things you cut, one of your pull requests in detail, and how you work with these tools day to day.

If you are hired, we pay for the six hours as part of your first month. Either way you hear from us within a week, with a reason.

QUESTIONS WE EXPECT

Answered in advance

Can I start from a boilerplate or a starter kit?	Yes. Name it in the README.
Do I need tests?	No. Say in one line what you would test first and why it was not the best use of hour five.
Can I swap Supabase for something else?	No. That constraint exists so submissions are comparable to each other.
Does it need to be deployed?	No. We run it locally. A deployed URL is welcome but earns nothing.
How finished should it look?	Rough edges are expected. A missing empty state is a rough edge. Three screens that do not connect is not.
How stubbed is too stubbed?	Hardcoding the current user is fine. Hardcoding what they may see is not.
What if my agent gets it right first time?	Then your review says so and you merge. We are not asking you to invent problems, only to show that you looked.
Six hours is not enough for this.	Correct. Choosing what fits is the exercise.

BEFORE YOU SEND IT

- ☐ **README:** clone to running in ten minutes, plus seed data, how to switch role, and your real hours
- ☐ Branches and **merged pull requests** with your review comments on them, unsquashed
- ☐ **DECISIONS.md**, two pages maximum
- ☐ Repository **public**, and the link in the proposal text itself